

UNIX-Programming

BASH

Unit-05

Unix programming and shell scripting like Bash are important for their ability to **automate tasks, process text, ensure portability, offer customization, and manage system resources efficiently**. These capabilities are essential for system administrators, developers, and anyone working with Unix-like operating systems.

Automation of Repetitive Tasks: With shell scripts, you can automate tasks such as file management, system monitoring, and batch processing, which saves time and reduces the potential for human error.

Powerful Text Processing Tools: Unix provides a suite of powerful text processing tools like `grep`, `awk`, and `sed`, which are indispensable for working with text data, logs, and configuration files. These tools allow for complex data manipulation and analysis, making it easier to extract, transform, and process data efficiently.

Portability and Compatibility: Unix scripts can be run on any Unix-like operating systems, such as Linux, macOS, and BSD variants, without the need for modification.

Customization and Control: Unix programming offers a high degree of customization and control over the system. Users can write scripts to modify their environment, manage system services, and configure system settings to their liking. This level of control allows users to optimize their workflows and system performance.

Foundation for Many Modern Technologies: Unix-like operating systems are the foundation for many modern technologies, including web servers, cloud computing, and mobile devices. Understanding Unix programming is essential for working with these technologies, as it provides the knowledge to manage and interact with the underlying systems effectively.

Efficient Resource Management: Unix programming allows users to directly manage system resources, such as processes and memory, through the command line. This direct access facilitates efficient resource management, enabling users to monitor and optimize system performance.

Scripting for Development and Deployment: For software developers, Unix scripting is crucial for automating development and deployment processes. Scripts can be used to automate code compilation, testing, and deployment, making the software development lifecycle more efficient.

Learning Unix Philosophy: The Unix philosophy emphasizes building simple, modular programs that do one thing well and can be combined to perform complex tasks. Learning Unix programming instills this philosophy, encouraging the creation of efficient, reusable code.

BASICS: Declaring Variable

Basic Variable Declaration

- **Without initializing (not common in Bash):**

bash

 Copy code

```
declare my_variable
```

- **With initialization:**

bash

 Copy code

```
my_variable="value"
```

BASICS: Declaring Variable

Integer Declaration

Using `declare` with `-i` makes Bash treat the variable as an integer. This allows arithmetic operations without needing to explicitly use `$(())` syntax.

bash

 Copy code

```
declare -i number
number=10+5
echo $number # Output will be 15, as arithmetic evaluation is done automatically
```

Arrays

Declare an indexed array using `-a`:

bash

 Copy code

```
declare -a myArray
myArray=(one two three four)
echo "${myArray[2]}" # Output will be 'three'
```

BASICS: Declaring Variable

Integer Declaration

Using `declare` with `-i` makes Bash treat the variable as an integer. This allows arithmetic operations without needing to explicitly use `$(...)` syntax.

bash

 Copy code

```
declare -i number
number=10+5
echo $number # Output will be 15, as arithmetic evaluation is done automatically
```

Arrays

Declare an indexed array using `-a`:

bash

 Copy code

```
declare -a myArray
myArray=(one two three four)
echo "${myArray[2]}" # Output will be 'three'
```

BASICS: Declaring Variable

Declare an associative array using `-A` (Note: Associative arrays are supported in Bash version 4 and above):

bash

 Copy code

```
declare -A capitals  
capitals["France"]="Paris"  
capitals["Germany"]="Berlin"  
echo "${capitals["Germany"]}" # Output will be 'Berlin'
```

BASICS: Declaring Variable

Read-only Variables

Making a variable read-only using `-r`. After this declaration, the variable cannot be modified or unset.

bash

 Copy code

```
declare -r myVar="Fixed Value"
echo $myVar # Output: Fixed Value
myVar="New Value" # This will raise an error: myVar: readonly variable
```

Exporting Variables to Environment

Variables can be exported to the environment of subsequently executed commands using `-x`.

bash

 Copy code

```
declare -x globalVar="I am visible to subprocesses."
```

This variable is now available in scripts and commands started from this shell.

BASICS: Declaring Variable

Making All References to a Variable Lowercase or Uppercase

- **Lowercase** `-l`: Converts the value to lowercase upon assignment.
- **Uppercase** `-u`: Converts the value to uppercase upon assignment.

bash

 Copy code

```
declare -l lowerVar="This WAS Mixed CASE."  
echo $lowerVar # Output will be 'this was mixed case.'
```

```
declare -u upperVar="This was mixed case."  
echo $upperVar # Output will be 'THIS WAS MIXED CASE.'
```

BASICS: Declaring Variable

What happens if
we run following
without
declaring?

```
echo $number
```

BASICS: Declaring Variable

set/unset vs empty variable

Characteristics of Empty Variables

- **No Assigned Value:** An empty variable is one that either has never been assigned a value or has been explicitly cleared.
- **Length of Zero:** When checked, an empty variable's length is zero. This can be tested using various scripting conditions.
- **Different from Unset:** An empty variable is distinct from an unset variable. An unset variable is one that has not been declared at all. In shell scripting, testing for emptiness and testing for being set can yield different results.

BASICS: Special Variables

Special Shell Variables

- ``$?`` - Exit status of the last command.
- ``$$`` - PID (Process ID) of the current shell.
- ``$0`` - Name of the script.
- ``$1, $2, ...`` - Positional parameters.
- ``$#`` - Number of positional parameters.
- ``$@`` or ``$*`` - All positional parameters.

Standard Exit Codes

- **0**: Success - The command executed successfully without any errors.
- **1**: General Error - A broad, generic error indicating that something went wrong, but the error is not categorized by any of the more specific non-zero exit codes.
- **2**: Misuse of Shell Builtins (according to Bash documentation) - This usually indicates that a shell builtin received incorrect arguments, but it can also be used by other commands to indicate more specific errors.

Common Exit Codes in Unix/Linux

Beyond the standard codes, the meaning of exit codes can vary between Unix commands and scripts. However, some common conventions include:

- **126**: Command invoked cannot execute - An error code indicating that the command cannot be executed, potentially due to permissions issues or because the file is not an executable.
- **127**: Command not found - Indicates that the command could not be found by the shell.

BASICS: Using and exporting Variables

Exporting Variables

- **Export to Environment (making it available to child processes):**

bash

 Copy code

```
export my_variable="visible in child processes"
```

Using Variables

- **Accessing Variables:**
 - General access: `\${variable\_name}`
 - Array element: `\${my\_array[0]}` (for the first element)
 - All elements of an array: `\${my\_array[@]}`
 - Keys of an associative array: `\${!my\_assoc\_array[@]}`

BASICS: Expressions Examples

Bash supports arithmetic operations and expressions directly within the shell, using the ``$((expression))`` syntax for arithmetic expansion. This feature enables performing basic mathematical operations without needing external programs like ``bc`` or ``awk`` for simple calculations. Here's a cheat sheet of examples showcasing how to use arithmetic expressions in Bash:

Basic Arithmetic Operations

- **Addition**

bash

 Copy code

```
result=$(( 5 + 3 )) # result: 8
```

- **Pre-Increment**

bash

 Copy code

```
a=1
(( ++a )) # a becomes 2
```

bash

 Copy code

```
a=5
(( a += 3 )) # a becomes 8
```

BASICS: Conditionals

bash

 Copy code

```
#!/bin/bash

# Check if the first argument to the script is greater than 10
if [ "$1" -gt 10 ]; then
    echo "The number is greater than 10."
elif [ "$1" -eq 10 ]; then
    echo "The number is exactly 10."
else
    echo "The number is less than 10."
fi
```

```
keshav@KESHAVBHAND8371 CLANGARM64
$ bash variable.sh 10
The number is exactly 10.
```

```
keshav@KESHAVBHAND8371 CLANGARM64
$ bash variable.sh 40
The number is greater than 10.
```

BASICS: Test Commands

In shell scripting, the ``[ ... ]`` construct is called the *test* command. It is used to evaluate conditional expressions and perform various checks, such as comparing strings, checking file attributes, and performing arithmetic comparisons. The ``[ ... ]`` provides a way to write conditional expressions in ``if``, ``while``, and other constructs that require a condition to be specified.

Example Usage

- **String comparison:**

bash

 Copy code

```
if [ "$a" = "$b" ]; then
    echo "Strings are equal."
fi
```

- **Arithmetic comparison:**

bash

 Copy code

```
if [ "$num1" -gt "$num2" ]; then
    echo "num1 is greater than num2."
fi
```

- **File existence check:**

bash

 Copy code

```
if [ -e "$filename" ]; then
    echo "File exists."
fi
```

Note on ``[[...]]`

BASICS: Comparison Operations

Numerical Comparison Operators

- ``-eq``: Equal to (``==`` in other languages)
- ``-ne``: Not equal to (``!=`` in other languages)
- ``-gt``: Greater than (``>`` in other languages)
- ``-ge``: Greater than or equal to (``>=`` in other languages)
- ``-lt``: Less than (``<`` in other languages)
- ``-le``: Less than or equal to (``<=`` in other languages)

File Test Operators

- ``-e``: File exists.
- ``-f``: File exists and is a regular file.
- ``-d``: File exists and is a directory.
- ``-r``: File exists and is readable.
- ``-w``: File exists and is writable.
- ``-x``: File exists and is executable.
- ``-s``: File exists and is not empty.
- ``-L``: File exists and is a symbolic link.

String Comparison Operators

- ``=`` or ``==``: Equal to. Checks if two strings are identical.
- ``!=``: Not equal to. Checks if two strings are not identical.
- ``<``: Less than. Checks if string1 sorts before string2 lexicographically in the current locale.
- ``>``: Greater than. Checks if string1 sorts after string2 lexicographically in the current locale.
- ``-z``: String is null, that is, has zero length.
- ``-n``: String is not null.

BASICS: Conditionals | more examples

bash

 Copy code

```
if [ -v variable ]; then
    echo "variable is set."
else
    echo "variable is unset."
fi
```

bash

 Copy code

```
if [ -z "$variable" ]; then
    echo "The variable is empty."
else
    echo "The variable is not empty."
fi
```

bash

 Copy code

```
if [ "$variable" = "" ]; then
    echo "The variable is empty."
else
    echo "The variable is not empty."
fi
```

BASICS: Conditionals | more examples

bash

 Copy code

```
if [ -v variable ]; then
    echo "variable is set."
else
    echo "variable is unset."
fi
```

bash

 Copy code

```
if [ -z "$variable" ]; then
    echo "The variable is empty."
else
    echo "The variable is not empty."
fi
```

bash

 Copy code

```
if [ "$variable" = "" ]; then
    echo "The variable is empty."
else
    echo "The variable is not empty."
fi
```

BASICS: Conditionals | more examples

Conditional Expressions

Arithmetic expressions can be used in conditions, with a result of `0` being false, and any non-zero result being true.

bash

 Copy code

```
a=5
```

```
b=2
```

```
if (( a > b )); then
```

```
    echo "a is greater than b"
```

```
fi
```

BASICS: For Loop

Basic Syntax

bash

 Copy code

```
for var in list; do  
    # commands  
done
```

BASICS: For Loop

bash

 Copy code

```
for (( i=0; i<5; i++ )); do  
    echo "Number: $i"  
done
```

BASICS: For Loop

Loop Over a Sequence of Numbers

- **Specified Range:**

bash

 Copy code

```
for i in {1..5}; do  
    echo "Number $i"  
done
```

- **Specified Range with Increment:**

bash

 Copy code

```
for i in {0..10..2}; do # Print even numbers between 0 and 10  
    echo "Even $i"  
done
```

BASICS: For Loop

Loop Over Strings

bash

 Copy code

```
for name in Alice Bob Charlie; do  
    echo "Hello, $name"  
done
```

BASICS: For Loop

Loop Over Strings

bash

 Copy code

```
for name in Alice Bob Charlie; do  
    echo "Hello, $name"  
done
```

BASICS: For Loop

Loop Over Array Elements

- Indexed Arrays:

bash

 Copy code

```
names=('Alice' 'Bob' 'Charlie')
for name in "${names[@]"; do
    echo "Hello, $name"
done
```

- Associative Arrays:

bash

 Copy code

```
declare -A fruits
fruits=([apple]='100' [banana]='50' [cherry]='75')

for fruit in "${!fruits[@]"; do
    echo "$fruit costs ${fruits[$fruit]} cents"
done
```

BASICS: For Loop

Loop Over Files and Directories

bash

 Copy code

```
for file in *; do
    echo "File name: $file"
done
```

Loop Over Command Output

bash

 Copy code

```
for user in $(cat users.txt); do
    echo "User name: $user"
done
```

Infinite Loop

bash

 Copy code

```
for (( ; ; )); do
    echo "This will run forever until stopped."
    sleep 1 # Sleep for 1 second to prevent overwhelming the terminal
done
```

BASICS: For Loop

Loop Control Statements

- `break`: Exits the loop immediately.
- `continue`: Skips the remaining commands in the loop body for the current iteration and proceeds with the next iteration of the loop.

bash

 Copy code

```
for i in {1..5}; do
  if [ "$i" -eq '3' ]; then
    continue # Skip number 3
  fi
  echo "Number $i"
done
```

BASICS: While Loop

bash

 Copy code

```
while [ condition ]; do  
    # Commands to execute  
done
```

BASICS: while-loop: Reading File examples

- `#!/bin/bash`: This is known as a shebang or hashbang. It specifies the path to the Bash interpreter that should be used to execute the script. This line tells the operating system to run this script with `/bin/bash`, ensuring it's interpreted as a Bash script regardless of the executing shell.
- `IFS=`: This temporarily sets the Internal Field Separator (IFS) to an empty value for the `read` command.
- `read -r line`: This command reads input from standard input (stdin). The `-r` option prevents backslash escapes from being interpreted, which means that backslashes are treated as literal characters.
- `do echo "$line"; done`: For each iteration of the loop, the script executes the `echo "$line"` command, which prints the content of the line.
- `< file.txt`: This redirects the contents of `file.txt` to the while loop.

bash

```
#!/bin/bash
```

```
while IFS= read -r line; do  
    echo "$line"  
done < file.txt
```

Note on IFS

bash

```
IFS=',' read -r first second third <<< "a,b,c"  
echo "First: $first, Second: $second, Third: $third"
```

BASICS: System Monitoring and Management

System Monitoring and Management

- **`top`**: Displays an ongoing view of system processes and resource usage. Great for real-time monitoring of CPU, memory, and process statistics.

```
bash
```

[Copy code](#)

```
top
```

- **`htop`** (an enhanced version of **`top`**): Provides a more user-friendly and visually appealing interface to monitor system processes.

```
bash
```

[Copy code](#)

```
htop
```

- **`free`**: Shows the total amount of free and used physical and swap memory in the system.

```
bash
```

[Copy code](#)

```
free -h # '-h' option for human-readable output
```

- **`df`**: Displays information about disk space usage on all mounted filesystems.

```
bash
```

[Copy code](#)

```
df -h # '-h' option for human-readable output
```

- **`du`**: Estimates file space usage for a directory.

```
bash
```

[Copy code](#)

```
du -sh /path/to/directory # '-s' for summary, '-h' for human-readable
```

BASICS: Process Management

Process Management

- **`ps`**: Reports a snapshot of current processes.

```
bash
```

[Copy code](#)

```
ps aux # 'a' for all users, 'u' for detailed user-oriented format, 'x' for processes without a controlling terminal
```

- **`kill`**: Sends a signal to a process, typically to terminate the process.

```
bash
```

[Copy code](#)

```
kill PID # Replace 'PID' with the process ID
```

- **`pkill`**: Allows killing processes by name, user, and other attributes.

```
bash
```

[Copy code](#)

```
pkill process_name
```

- **`nohup`**: Runs a command immune to hangups, with output to a non-tty.

```
bash
```

[Copy code](#)

```
nohup command &
```

BASICS: Networking

Networking

- **`ping`**: Checks the network connectivity to another host.

```
bash
```

[Copy code](#)

```
ping hostname_or_ip
```

- **`ifconfig`** or **`ip`**: Displays or configures network interface settings.

```
bash
```

[Copy code](#)

```
ifconfig # or 'ip addr show'
```

- **`netstat`**: Displays network connections, routing tables, interface statistics, masquerade connections, and multicast memberships.

```
bash
```

[Copy code](#)

```
netstat -tuln # Options: 't' for TCP, 'u' for UDP, 'l' for listening, 'n' for numer
```

- **`ssh`**: Secure Shell, a protocol for securely logging into remote machines. The command initiates a secure shell session.

```
bash
```

[Copy code](#)

```
ssh user@host
```

BASICS: File Transfers

File Transfer and Backup

- **`rsync`**: An advanced tool for copying files and directories between locations with options for compression, recursion, and synchronization.

bash

 Copy code

```
rsync -avz source/ destination/ # 'a' for archive mode, 'v' for verbose, 'z' for co
```

- **`tar`**: An archiving utility to create, modify, or extract files from a tarball archive (**` .tar`**, **` .tar.gz`**, **` .tar.bz2`**, etc.).

bash

 Copy code

```
tar -czvf name-of-archive.tar.gz /path/to/directory/ # Create a compressed archive  
tar -xzvf name-of-archive.tar.gz -C /path/to/destination # Extract an archive
```

BASICS: System Information

System Information

- **`uname`**: Displays system information.

```
bash
```

[Copy code](#)

```
uname -a # '-a' displays all information
```

- **`lscpu`**: Displays information about the CPU architecture.

```
bash
```

[Copy code](#)

```
lscpu
```

- **`lsblk`**: Lists information about all available or the specified block devices.

```
bash
```

[Copy code](#)

```
lsblk
```

BASICS: Web Service: WGET & CURL
(self explore... out of scope)

BASICS: Web Service: WGET & CURL
(self explore... out of scope)

GREP: Powerful Command line utility

grep is a command-line utility in Unix and Linux used for searching plain-text data sets for lines that match a regular expression. Its name comes from the ed command **g/re/p** (**g**lobally **s**earch a **r**egular **e**xpression and **p**rint). Here's a cheat sheet covering common grep usage scenarios:

Basic `grep` Usage

- Search for a pattern within a file:

```
bash
```

```
grep "pattern" filename
```

- Search for a pattern in multiple files:

```
bash
```

```
grep "pattern" file1 file2 file3
```

- Case-insensitive search:

```
bash
```

```
grep -i "pattern" filename
```

- Count the number of lines that match the pattern:

```
bash
```

```
grep -c "pattern" filename
```

Regular Expressions and Patterns

- Search for lines that match one of multiple patterns (use `-e` for each pattern):

```
bash
```

[Copy code](#)

```
grep -e "pattern1" -e "pattern2" filename
```

- Use extended regular expressions (more powerful syntax):

```
bash
```

[Copy code](#)

```
grep -E "pattern" filename
```

Context Control

- Print only matched parts of a line, instead of entire lines:

```
bash
```

[Copy code](#)

```
grep -o "pattern" filename
```

GREP: Powerful Command line utility.... more

Line Number and File Name Display

- Show line numbers for matching lines:

bash

 Copy code

```
grep -n "pattern" filename
```

- Only print the names of files with matching lines, not the matching lines themselves:

bash

 Copy code

```
grep -l "pattern" /path/to/directory/*
```

Using `grep` with Pipes

- `grep` can be used with pipes to search within the output of another command:

bash

 Copy code

```
ls /path/to/directory | grep "pattern"
```

GREP: Powerful Command line utility.... more

File and Directory Searches

- Recursively search for a pattern in all files under a directory:

bash

 Copy code

```
grep -r "pattern" /path/to/directory
```

- Search for a pattern in all files of a specific type:

bash

 Copy code

```
grep -r --include="*.txt" "pattern" /path/to/directory
```

- Exclude directories from a recursive search:

bash

 Copy code

```
grep -r --exclude-dir={dir1,dir2} "pattern" /path/to/directory
```

GREP: Powerful Command line utility.... more

Output Control

- Display lines that do *not* match the pattern:

bash

 Copy code

```
grep -v "pattern" filename
```

- Display N lines after the match:

bash

 Copy code

```
grep -A N "pattern" filename
```

- Display N lines before the match:

bash

 Copy code

```
grep -B N "pattern" filename
```

- Display N lines around the match:

bash

 Copy code

```
grep -C N "pattern" filename
```

GREP: Powerful Command line utility.... more

bash

 Copy code

```
multi_line_variable="This is the first line.  
This is the second line with the keyword.  
This is the third line."
```

```
matches=$(echo "$multi_line_variable" | grep "keyword")  
echo "Matches: $matches"
```

GREP: Extended Regular Expressions

Operators in Extended Regular Expressions

- **Alternation** (`|`): Matches either the expression before or the expression after the operator. For example, `grep -E 'cat|dog'` matches lines containing `cat` or `dog`.
- **Grouping** (`()`): Groups expressions as a single unit for alternation or to apply quantifiers. For example, `grep -E '(cat|dog)house'` matches `cathouse` or `doghouse`. In BRE, you would need to escape the parentheses as `\(` and `\)`.
- **Quantifiers:**
 - `?`: Matches 0 or 1 occurrence of the preceding element. For example, `grep -E 'cats?'` matches `cat` or `cats`.
 - `+`: Matches 1 or more occurrences of the preceding element. For example, `grep -E 'cat+'` matches `cat`, `catt`, `cattt`, etc.
 - `{n}`: Matches exactly `n` occurrences of the preceding element.
 - `{n,}`: Matches `n` or more occurrences.
 - `{n,m}`: Matches between `n` and `m` occurrences, inclusive.

In BRE, the quantifiers `?`, `+`, and `{}` are not available, or you need to escape them with a backslash (`\?`, `\+`, `\{n,m\}`).

- **Anchors:**
 - `^`: Matches the start of a line.
 - `$`: Matches the end of a line.

Anchors work the same way in both BRE and ERE.

GREP: Extended Regular Expressions

Examples of ERE Patterns

1. **Matching phone numbers** (assuming a format of `XXX-XXX-XXXX`):

regex

 Copy code

```
grep -E '^[0-9]{3}-[0-9]{3}-[0-9]{4}$' file.txt
```

2. **Matching email addresses** (a simple pattern):

regex

 Copy code

```
grep -E '^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,4}$' file.txt
```

3. **Matching words with optional endings:**

regex

 Copy code

```
grep -E 'root(s|ed|ing)?' file.txt
```

4. **Complex pattern with alternation:**

regex

 Copy code

```
grep -E '(error|fail|fatal)' log.txt
```

GREP: Functions

bash

 Copy code

```
function_name() {  
    echo "First argument: $1"  
    echo "Second argument: $2"  
}
```

```
function_name "arg1" "arg2"
```


Some helpful stuff...

bash

 Copy code

```
# Declare an empty array
my_list=()

# Append items to the array
my_list+=(item1)
my_list+=(item2)
my_list+=(item3)

# Alternatively, you can append multiple items at once
my_list+=(item4 item5 item6)

# Print the contents of the array
echo "${my_list[@]}"
```

- ``my_list+=(item)`` appends ``item`` to the end of the array ``my_list``.
- ``my_list+=(item1 item2 item3)`` appends multiple items to the array ``my_list``.
- ``"${my_list[@]}"`` expands to all the elements of the array ``my_list``.

Some helpful stuff...

Using `unset` to Remove a Specific Element from an Array

bash

 Copy code

```
# Declare an array
my_list=("item1" "item2" "item3" "item4")

# Remove "item2" from the array
unset 'my_list[1]'

# Print the contents of the array
echo "${my_list[@]}"
```

This will output the contents of the array `my_list` after removing "item2".